

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

Ch. 001

Introduction of C Language

What is a Language?

- Language is a collection of word and symbols.
- Language is generally used to build up a communication between persons, machines or computers.
- Generally human beings used different languages in different region for effective communication.
- Languages is used to give some instructions / information / commands / details to other person.
- Basically, computer is a machine and machine cannot do any thing without instruction/command.
- If we want to work with computer and with to do some work as per our requirement then we must give some instructions / command
- Generally computer understand some special languages only. These languages are known as computer languages.

There are mainly three different languages...

1. Low Level Languages
2. High Level Languages
3. Middle Level Languages

1. Low level programming language :

- Low level programming language is also known as Binary Language too.
- Binary Language consists only two characters **0 [Zero]** and **1 [One]**.
- Low level programming language does not require translator like compiler or interpreters.
- Machine can direct understand Low level programming with fast execution speed.

2. High level programming language :

- High level languages are user friendly language because their instructions are similar to human languages.
- High level languages have a set of grammar that makes it easy for a programmer to write programs and identify the correct errors in a program.
- All high level languages have its own compiler or interpreter to convert its code into machine code (Binary Codes).

For Examples,

Cobol, Fortran, Basic, Java etc.

4. Middle level programming language :

- Middle Level Programming is a language which has mixed power of Low Level Programming Language and High Level Programming Language.
- Middle Level Programming is Easy for Programming compare to Low Level Programming.
- Middle Level Programming is fast for execution compare to High Level Programming.

❖ What is POP?

POP is a procedure oriented programming language.

C, Basic and Fortran are example of POP.

What is translation?

Translation means to convert source-language by target-language.

Examples :

1. Gujarati **To** Hindi
2. Gujarati **To** English
3. English **To** Gujarati
4. Text **To** Binary
5. Binary **To** Text

❖ What is translator?

Translator is a special program. This translator accepts the user program and checks each statement and produces a set of Machine language instructions.

There are two types of Translators.

- **Compiler**
- **Interpreters**

What is Compiler?

- A Compiler checks all the entire program written by user
- If program is free from error and mistakes then compiler produces this program as a complete program.
- If program founds some errors in the program then it does not execute a single statement of the program.
- So Compiler translate whole program in Machine language before it starts execution.

What is Interpreters?

- Interpreters perform the similar job like compiler but in different way.
- Interpreters translates one statement at a time and if it is error-free then executes that statement.
- This continues till the last statement in the program has been translated and executed.

Difference between Compiler & Interpreter?

Compiler	Interpreter
Compiler first checks all the statement for error and provide lists of all the errors in the program.	Error finding is much easier in Interpreter because it checks and executes each statement at a time.
Compiler take less time than interpreter because it translate and executes all statement at same time.	Interpreter take more time for the execution of a program compared to Compilers because it translates and executes each statement one by one.

Introduction to C : History Of C:

It was developed by step as given below.

Language	Year	Developed by
ALGOL (ALGOarithmic Language)	1960	International Committee
CPL (Combined Programming Language)	1963	Cambridge University
BPCL (Basic Combined Programming Language)	1967	Martin Richards & Cambridge University
B (B Language)	1970	Ken Thomson at AT&T Laboratory
Traditional C (C Language)	1972	Denis Ritchie at AT&T Laboratories
ANSI C (American National Standards Institute)	1989	ANSI Committee
C99	1999	Standardization Committee

What is C?

- ✓ C is programming language.
- ✓ It is also known as middle level programming language.
- ✓ Program execution speed is faster than High Level Programming Languages.
- ✓ C behaves as High Level Language through functions and reusability
- ✓ C programming is faster than other programming languages.
- ✓ C is a powerful and flexible language.
- ✓ C program has its own built in library functions
- ✓ Write C program with UDF (User Defined Function) makes program more simple and easy to understand.
- ✓ The C Language is developed by Dennis Ritchie for creating system applications that directly interact with the hardware devices such as drivers, kernels, etc.
- ✓ C programming is considered as the base for other programming languages, that is why it is known as mother language.

It can be defined by the following ways:

1. Mother language
2. System programming language
3. Procedure-oriented programming language
4. Structured programming language
5. Mid-level programming language

Shortcut Used in C editor

:

What to DO ?	Shortcut
Save a program file	F2
Open a saved file	F3
To maximize editing window	F5
Undo last changes in Editor	Alt+BackSpace
Redo last changes by Undo	Shift+Alt+B.Space
Copy selected text	Ctrl + Insert Key
Cut selected text	Shift + Del
Paste copied/cut text	Shift + Insert Key
Clear Selected Text	Ctrl + Del
Compile a C Program	Alt + F9
Run a C Program	Ctrl + F9
Step by Step Debugging	F7

Structure of C Program :

Documentation Section

File include section (Link Section)

Define Symbolic Constant

Global declaration Section

Function Prototype

main() (Main Function Section)

{ declaration part

executable part

}

User defined Function or Sub program

First C Program :

// write a program to print Hello Creative

`#include <stdio.h>`

`int main()`

{

`printf("Hello Creative");`

`return 0;`

}

#include <stdio.h> includes the standard input output library functions. The **printf()** function is defined in `stdio.h`.

int main() The `main()` function is the entry point of every program in c language.

return 0 The `return 0` statement, returns execution status to the OS. The 0 value is used for successful execution and 1 for unsuccessful execution.

Assignment Work:

- 1)Write a Program to Print your Bio-data using Printf Function
- 2) WAP to print Student Result Format using Printf function

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

Ch. 002

Data Type & Variable

C Tokens :

- Identifiers
- Keywords
- Constants
- Operators
- Strings and special symbols, these are called as C tokens.

Identifiers :

Identifier is a name of variable, constant, structure, union, arrays or function given by the programmer.

Keywords :

- Keyword is a word which is having a specific meaning for a particular language.
- Keywords are reserved word by programming language.
- Keywords are not used as identifiers.
- There are 32 keywords available in C.

auto	break	case	char
const	continue	do	Double
default	else	enum	extern
float	for	goto	if
int	long	register	return
struct	static	short	signed
switch	sizeof	typedef	union
unsigned	void	volatile	while

Variables :

- A variable is a name of the memory location. It is used to store data. Its value can be changed, and it can be reused many times.
- It is a way to represent memory location through symbol so that it can be easily identified.
- Let's see the syntax to declare a variable:

```
int a;
float b;
char c;
```

Rules for variable naming :

- The first character in identifier name must be an alphabet or underscore
- commas or blanks space are not allowed within an identifier name.
- special symbol other than an underscore can be not used as in identifier name.
- An identifier name is any combination of 1 to 31 alphabets, digits or underscores without space.
- Do not create unnecessarily long variable names as it adds to your typing efforts.
- C language is case sensitive. Upper and Lower case both are allowed in C programming. But they are not equivalent. Here **NUM** and **num** both are different.
- A variable name must not be any reserved word or keyword, e.g. int, float, etc.

Types of Variables in C :

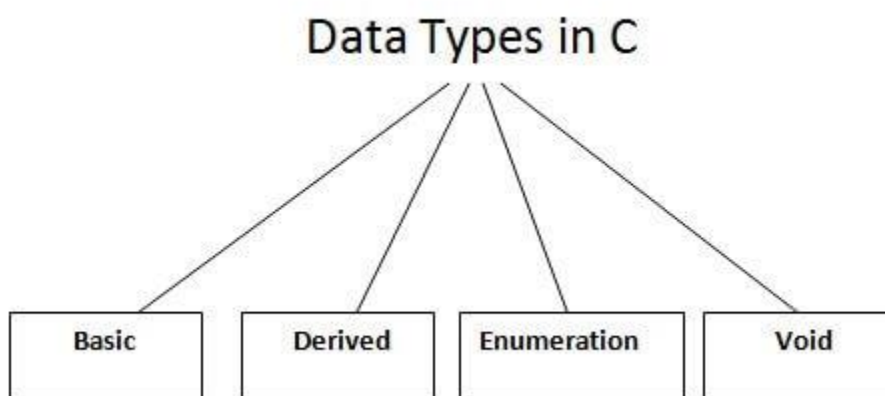
There are many types of variables in c:

1. local variable
2. global variable
3. static variable
4. automatic variable
5. external variable

All types of variable we will learn in ch - 21

Data Types in C :

A data type specifies the type of data that a variable can store such as integer, floating, character, etc.



There are the following data types in C language.

Types	Data Types
Basic Data Type	int, char, float, double
Derived Data Type	array, pointer, structure, union
Enumeration Data Type	enum
Void Data Type	void

Basic Data Types :

The basic data types are integer-based and floating-point based. C language supports both signed and unsigned literals.

The memory size of the basic data types may change according to 32 or 64-bit operating system.

Let's see the basic data types. Its size is given **according to 32-bit architecture**.

Data Types	Memory Size	Format code	Range
char	1 byte	%c	-128 to 127
int	2 byte	%d	-32,768 to 32,767
unsigned int	2 byte	%u	0 to 65,535
long int	4 byte	%ld	-2,147,483,648 to 2,147,483,647
float	4 byte	%f	-2,147,483,648 to 2,147,483,647
double	8 byte	%lf	

Integer :

- ✓ This data type is used to store numbers without decimals.
- ✓ Range between -32768 to +32767
- ✓ Occupies 2 byte (16 bits)
- ✓ Format code is %d

Example,

```
int a,b,c;
a=10;
b=20;
c=a+b;
```

printf() and scanf() in C :

The printf() and scanf() functions are used for input and output in C language. Both functions are inbuilt library functions, defined in stdio.h (header file).

printf() function :

The printf() function is used for output. It prints the given statement to the console.

The syntax of printf() function is given below:

printf("format string",argument_list);

The format string can be %d (integer), %c (character), %s (string), %f (float) etc.

scanf() function :

The scanf() function is used for input. It reads the input data from the console.

scanf("format string",argument_list);

Program to print cube of given number

Let's see a simple example of c language that gets input from the user and prints the cube of the given number.

```
#include<stdio.h>
int main(){
int number;
printf("enter a number:");
scanf("%d",&number);
printf("cube of number is:%d ",number*number*number);
return 0;
}
```

Output :

```
enter a number:5
cube of number is:125
```

The scanf("%d",&number) statement reads integer number from the console and stores the given value in number variable.

The printf("cube of number is:%d ",number*number*number) statement prints the cube of number on the console.

Program to print sum of 2 numbers :

Let's see a simple example of input and output in C language that prints addition of 2 numbers.

```
#include<stdio.h>
int main(){
int x=0,y=0,result=0;

printf("enter first number:");
scanf("%d",&x);
printf("enter second number:");
scanf("%d",&y);

result=x+y;
printf("sum of 2 numbers:%d ",result);

return 0;
}
```

Output :

```
enter first number:9
enter second number:9
sum of 2 numbers:18
```

Assignment Work :

1) WAP to declare two variable and calculate (sum/sub/mul/div)using this variable

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

Ch. 3

Unsigned | Long | Float | Char Data Type

Signed int:

- ❑ By default all the data types are signed. So it is not necessary to define any variable as signed.

■ Example :

```
signed int a=20;
```

```
int b=30;
```

```
int c=a+b;
```

```
clrscr();
```

```
printf("%d",c);
```

Unsigned int

- ❑ If we want to store only positive values in variable, then it is very easy by using **unsigned**.
- ❑ If we define any variable as unsigned the positive range will be increased of that variable by negative value.

■ Example :

- ❑ **integer** value will store **-32768 to +32767** in signed

- ❑ But if we will convert it in to **unsigned** it will store directly **0 to +65535 positive value**.

- ❑ **Format code %u**

Example :

```
Unsigned int a=25000,b=25000,c;
```

```
C=a+b;
```

```
Printf("C=%d",c);
```

Output :

```
C=50000
```

Long Int :

- A long int typically uses twice as many bits as a regular int, allowing it to hold much larger numbers.
- printf and scanf replace with %ld to indicate the use of a long int.
- long int allocate memory 4 bytes in c prog

Example :

```
long int a=200000,b=300000,c;
c=a+b;
printf("C=%ld",c);
```

output :

```
c=500000
```

Float :

- ✓ This data type is used to store numbers with decimals.
- ✓ Value without decimal can also be stored in it.
- ✓ Range between $3.4e - 38$ to $3.4e +38$
- ✓ Occupies 4 byte (32 bits)
- ✓ Format code is %f

Example,

```
float a,b,c;
a=10.50;
b=20.3;
c=a+b;
output:
c=30.8
```

Character :

This data type is used to store alphabets, number and special symbols in it.

Range between -128 to +127 (Range in ASCII)

Occupies 1 byte (8 bits)

Format code is %c

Example,

```
Char a='z';
```

Double :

- This data type is used to store numbers with decimals when float is insufficient.
- Range between $1.7e - 308$ to $1.7e +308$
- Occupies 8 byte (32 bits)
- Format code is %lf

Example,

```
double a,b,c;
a=9990990.90;
b=20.50;
c=a+b;
```

Void :

- ❑ Actually there is no variable defined of this type but it is used to define function.
- ❑ Void is data type which specifies that the function does not return any value to the calling program.

■ Example,

```
void main( )
void printnum( )
```

Type Casting :

- Type casting is a process which converts a value from one data type into another data type.
- During the programming **we can convert integer value to float value and float value to integer value** as per requirement.

■ Example :

```
int a=20;
printf("%f",(float)a/3); // It will convert data from integer to float.
```

Assignment Work :

1. WAP to Calculate Area and Circumference of circle (Area of Circle = $\text{PI} * R^2$)
Circumference of Circle = $2 * \text{PI} * R$)
2. WAP to Calculate Area of Square (area = side * side)
3. WAP to Calculate Area of Rectangle (area = l * b)

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

Ch. 4

Introduction of Operator (Arithmetic | Assignment)

What is Operator?

- Operators are the foundation of any programming language. Thus the functionality of C/C++ programming language is incomplete without the use of operators.
- We can define operators as symbols that help us to perform specific mathematical and logical computations on operands.
- In other words, we can say that an operator operates the operands.
- Like, **+, -, *, /, &&, ||, !=, <, >, etc.**

What is Operand?

Operand means the variable or value which is used before and after the operator.

Like, A+B (Here, A and B are operand and + is operator)

Operator :

- (1) Arithmetic Operator
- (2) Assignment Operator
- (3) Relational Operator
- (4) Logical Operator
- (5) Increment / Decrement Operator
- (6) Conditional Operator / Ternary Operator
- (7) Bitwise Operator
- (8) Special Operator

1. Arithmetic Operator :

An arithmetic operator performs mathematical operations such as addition, subtraction, multiplication, division etc on numerical values (constants and variables).

Operator	Purpose
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Reminder after integer division

Example :

1. WAP to declare two variable and calculate sum , sub, mul, div.
2. WAP to declare two variable and enter value of the variable and calculate Sum, sub,mul,div .

```
// Working of arithmetic operators
#include <stdio.h>
int main()
{
    int a = 9,b = 4, c;

    c = a+b;
    printf("a+b = %d \n",c);
    c = a-b;
    printf("a-b = %d \n",c);
    c = a*b;
    printf("a*b = %d \n",c);
    c = a/b;
    printf("a/b = %d \n",c);
    c = a%b;
    printf("Remainder when a divided by b = %d \n",c);

    return 0;
}
```

2. Assignment Operators:

Assignment operators are used to assign value to a variable. The left side operand of the assignment operator is a variable and right side operand of the assignment operator is a value. The value on the right side must be of the same data-type of variable on the left side otherwise the compiler will raise an error.

Different types of assignment operators are shown below:

“=”: This is the simplest assignment operator. This operator is used to assign the value on the right to the variable on the left.

For example:

```
a = 10;
```

```
b = 20;  
ch = 'y';
```

“+=”: This operator is combination of ‘+’ and ‘=’ operators. This operator first adds the current value of the variable on left to the value on right and then assigns the result to the variable on the left.

Example:

(a += b) can be written as (a = a + b)

Assignment Work :

1. WAP to find the simple interest (**$SI = (P \times R \times N) / 100$**)
2. WAP Program to Convert temperature from degree centigrade to Fahrenheit(**$Fahrenheit = (Celsius \times 9 / 5) + 32$**)
3. WAP to swap 2 value without using third variable
4. WAP to swap 2 value with third variable

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE
DESIGN & MULTIMEDIA INSTITUTE

Ch. 5

Relational Operator

Logic Development

Control Statement

3 Relational Operator :

- A relational operator checks the relationship between two operands. If the relation is true, it returns 1; if the relation is false, it returns value 0.
- Relational operators are used in decision making and loops.
- Relational Operators are return Boolean value like true and false

Operator	Meaning
<	Less than
<=	Less than or Equal to
>	Greater than
>=	Greater than or Equal to
= =	Equal to
!=	Not Equal to

Example :

```
Int a=5,b=2,c;
```

```
C=a>b;
```

```
Printf("\nC=%d",c);
```

Introduction of Logic :

- Logic is must to perform any task.
- Using various types of logic we can perform many programs.
- There are three types of Logic structures are possible.

☐ **Sequence Logic**

☐ **Decision Logic**

☐ **Looping Logic**

■ Sequence Logic.

- ❑ In this logic all the instructions are written in order to performed in a sequence.

■ Decision Logic

- ❑ When more then one option is available then we can use decision logic with.

■ If... Then

■ If... Then...else

■ If... elseif... else... endif

■ Looping Logic :

- ❑ When one or more instructions may be executed more then one time then we can use looping logic.

Instructions for Logic Development :

- To develop a program easily, we must have sufficient knowledge of Pre Programming Techniques.
- Pre Programming Techniques will help you to design a proper programming channel.
- If we are going to write a computer program using any programming language, we need to write a rough program first.
- There are two basic tools do develop basic or rough program.
 - Algorithm
 - Flowchart
- If we want to solve any error from any program the we must know the proper steps involved in solving problem.
- If the sequence of programming command is not proper then you will get wrong output.
- To solve these problems we can use Algorithm and Flowchart.

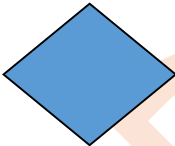
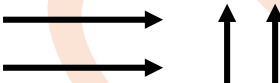
■ What is Algorithm?

- Write program solution step by step in textual form is known as algorithm.

■ What is Flowchart?

- A flowchart is a graphical representation of an algorithm.

Flowchart Symbols :

	Terminal Symbol (Start / Stop) This oval shape represents for Start and Stop.
	Process Steps Symbol The Rectangle represents the processing operation.
	Input or Output Process Symbol This shape represents input and output task.
	Decision Making and Branching Symbol The diamond shape represent decision or branching statements.
	Flow Direction Symbol The arrows shows flow direction in which one has to proceed to solve the problem.

Algorithm :

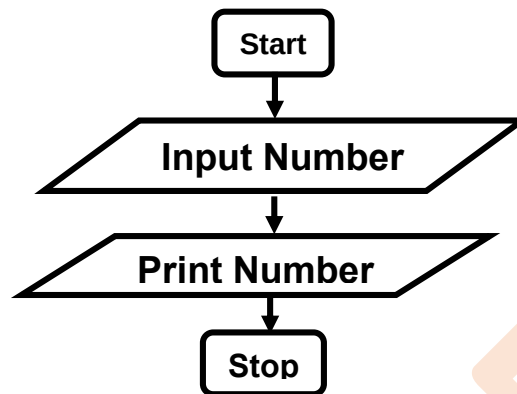
Write an algorithm and draw a flowchart to read and print value.

Step-1 Input the number

Step-2 Print the number

Step-3 Stop

Flowchart : Draw a flowchart to read and print value.



Algorithm :

2. Write an algorithm to make sum of three value.

Step-1 Input value1, value2, value3

Step-2 Calculate sum of three values $\text{Sum} = \text{value1} + \text{value2} + \text{value3}$

Step-3 Print sum of value

Step-4 Stop

Sequence Logic :

1. Algorithm Example :

Write an algorithm to make sum of three value.

Step-1 Input value1, value2, value3

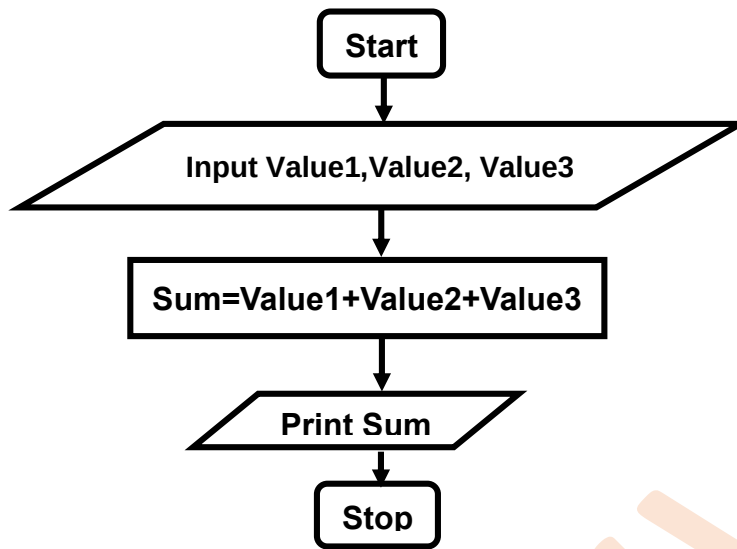
Step-2 Calculate sum of three values // $\text{Sum} = \text{value1} + \text{value2} + \text{value3}$

Step-3 Print sum of value

Step-4 Stop

Flowchart example:

Draw a flowchart to make sum of three value.



Control Statement :

■ There are two type of Control structure

- Conditional
- Looping

- ✓ In conditional statement there are optional environment available.
- ✓ Example : to watch movie or to play cricket
- ✓ If Statement
- ✓ Switch Statement
- ✓ If Statement in Details

□ We can write conditional statement IF in different ways...

- if (Wimple if Without Else Statement)
- if....else
- Nested if (if within if)
- if....else if ladder

if (Simple if without else) :

Syntax : if(condition)

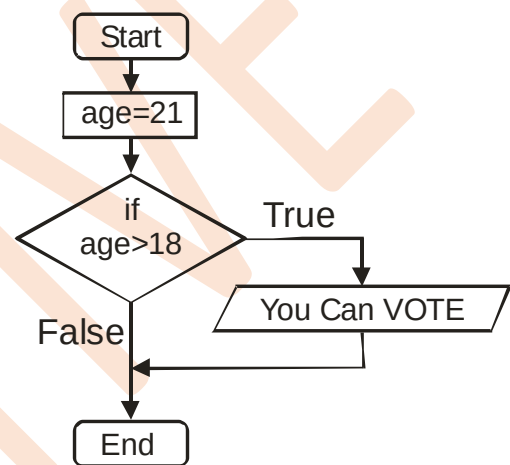
```
{
    statements;
}
```

■ Purpose:

- If statement is the decision control statement. This statement will check the given condition. If condition is TRUE then it execute given statements.

Example: To check that you can VOTE or Not

```
void main( )
{
    int age;
    printf("Enter your age :");
    scanf("%d",&age);
    if(age>18)
        printf("You can VOTE");
}
```



if....else :

Syntax :

```
if(condition)
{
    statement Group1;
}
Else
{
    statement Group2 ;
}
```

■ Purpose:

- If..else statement is a decision control statement.
- If condition is TRUE then it execute given statements group 1. If condition is FALSE then it executes statement group 2.

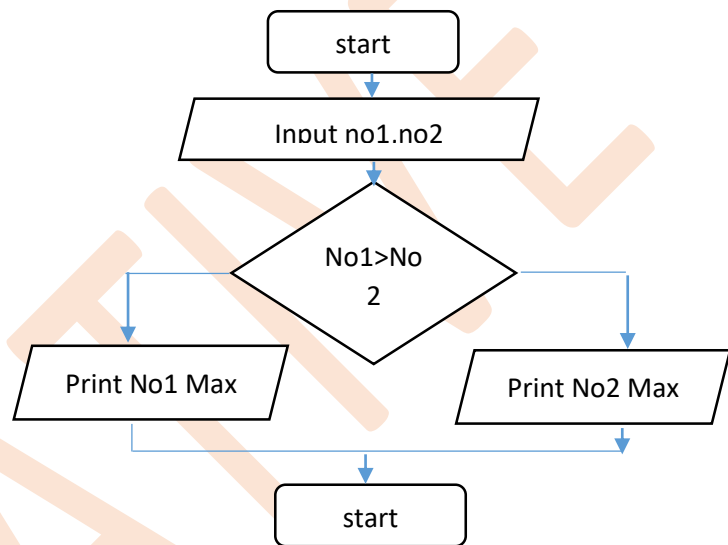
Example: To check that you can VOTE or Not

```
void main( )
{
    int age;
    printf("Enter your age :");
    scanf("%d",&age);
    if(age>18)
        printf("You can VOTE");
    else
        printf("You Can Not VOTE");
}
```

Draw a flowchart to find the maximum number from given two number.

Example :

```
void main( )
{
    int no1,no2;
    Printf("Enter No1);
    Scanf("%d",&no1);
    Printf("Enter No2);
    Scanf("%d",&no1);
    if(no1>no1)
        printf("No1 max");
    else
        printf("No2 max");
}
```



Assignment Work :

1. Draw a Flow chart to Find the Maximum/ Minimum number of two value
2. Draw a Flow chart to Find the Maximum/ Minimum number among the three number
3. Draw a Flow chart to Find the Maximum/ Minimum number among the four number
4. Draw a Flow chart to Find the Maximum/ Minimum number among the Five number
5. Draw a Flow char to check equal / min / max among the three value
6. WAP to Find the Maximum/ Minimum number of two value
7. WAP to Find the Maximum/ Minimum number among the three number
8. WAP to Find the Maximum/ Minimum number among the four number
9. WAP to Find the Maximum/ Minimum number among the Five number
10. WAP to check equal / min / max among the three value
11. WAP to check num and print **CREATIVE** if given num is greater than 10 or print **MULTIMEDIA** if given num is smallest 10 among three variable without logical operator
12. WAP to program to check whether the given number is positive ,negative or zero

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

Ch. 6

Logical Operator

Ladder else if

4 Logical Operators:

They are used to combine two or more conditions

Logical AND operator: The '&&' operator returns true when both the conditions under consideration are satisfied. Otherwise it returns false. For example, a && b returns true when both a and b are true (i.e. non-zero).

Logical OR operator: The '|' operator returns true even if one (or both) of the conditions under consideration is satisfied. Otherwise it returns false. For example, a | b returns true if one of a or b or both are true (i.e. non-zero). Of course, it returns true when both a and b are true.

Logical NOT operator: The '!' operator returns true the condition in consideration is not satisfied. Otherwise it returns false. For example, !a returns true if a is false, i.e. when a=0.

Example :

```
Int a=6,b=3,c=3;

If(a>b && a>c)
{
    Printf("a is max");
}else if(b>c)
{
    Printf("b is max");
}else
{
    Printf("c is max");
}
```

The if-else-if Ladder :

Syntax :

```
if(condition1)
    statement block1;
else if(condition2)
    statement block2;
else if(condition3)
    statement block3;
else
    statement block n;
```

Purpose:

Here in this form, condition1 is evaluated first, if it is true then the statement block 1 will be executed. If it is false then condition2 will be evaluated, if it is true then the statement block 2 will be executed. This will happen until one condition will be true. If all the conditions are false then the else part will be executed.

Assignment Work :

1. WAP to Find the Maximum/ Minimum number among the Five number using Logical operator
2. WAP to print Full Student Result of FIVE sub marks and calculate Total , per , min, max, Result, Grade
3. WAP to input electricity unit charges and calculate total electricity bill according to the given condition:
For first 50 units Rs. 0.50/unit /For next 100 units Rs. 0.75/unit
/For next 100 units Rs. 1.20/unit / For unit above 250 Rs. 1.50/unit /An additional surcharge of 20% is added to the bill

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

Ch. 7

**Increment and Decrement Operator
Turnery Operator | Modulus Operator**

5. Increment and Decrement Operator in C :

Increment Operators are used to increase the value of the variable by one and Decrement Operators are used to decrease the value of the variable by one in C programs.

Both increment and decrement operator are used on a single operand or variable, so it is called as a unary operator. Unary operators are having higher priority than the other operators it means unary operators are executed before other operators.

Syntax :

`++` // increment operator

`--` // decrement operator

Note: Increment and decrement operators are can not apply on constant.

Example

`x= 4++;` // gives error, because 4 is constant

Type of Increment Operator :

- pre-increment
- post-increment

pre-increment (++ variable) :

In pre-increment first increment the value of variable and then used inside the expression (initialize into another variable).

Syntax :

`++ variable;`

Example pre-increment :

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int x,i;
    i=10;
    x=++i;
    printf("x: %d",x);
    printf("i: %d",i);
    getch();
}
```

Output :

x: 11

i: 11

In above program first increase the value of i and then used value of i into expression.

post-increment (variable ++)

In post-increment first value of variable is used in the expression (initialize into another variable) and then increment the value of variable.

Syntax :

variable ++;

Example post-increment

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int x,i;
    i=10;
    x=i++;
    printf("x: %d",x);
    printf("i: %d",i);
    getch();
}
```

Output

x: 10

i: 11

In above program first used the value of i into expression then increase value of i by 1.

Type of Decrement Operator:

- pre-decrement
- post-decrement

Pre-decrement (-- variable) :

In pre-decrement first decrement the value of variable and then used inside the expression (initialize into another variable).

Syntax :

-- variable;

Example pre-decrement

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int x,i;
    i=10;
    x=--i;
    printf("x: %d",x);
    printf("i: %d",i);
    getch();
}
```

Output

x: 9

i: 9

In above program first decrease the value of i and then value of i used in expression.

post-decrement (variable --):

In Post-decrement first value of variable is used in the expression (initialize into another variable) and then decrement the value of variable.

Syntax :
variable --;

Example post-decrement

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int x,i;
    i=10;
    x=i--;
    printf("x: %d",x);
    printf("i: %d",i);
    getch();
}
```

Output

x: 10
i: 9

In above program first used the value of x in expression then decrease value of i by 1.

Example of increment and decrement operator

Example:

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int x,a,b,c;
    a = 2;
    b = 4;
    c = 5;
    x = a-- + b++ - ++c;
    printf("x: %d",x);
    getch();
}
```

Output
x: 0

6. Conditional or Ternary Operator (?:)

- C programming offers the conditional operator used in C language is (?) question mark and (:) column.
- It is used for checking the condition and gives output according to the condition.
- If condition is true then it will execute the first value or it will execute the second value given in the syntax.

Syntax: **<Condition>? <True Value> :< False Value>**

Purpose:

If condition will be true then it will return the true value else it will return the false value.

Example:

```
a=10;
b=12;
c= (a>b)? A: b      // It will store value of B
printf("Value of C is %d",c);

sub1=23, sub2=40;

Printf ("Highest=%d", (sub1>sub2)? sub1:sub2)
```

Modulus operator in C :

The modulus operator is a symbol used in various programming languages. It is denoted by the percentage symbol (%). It is a modulus operator that is used in the arithmetic operator. It determines the remainder. In some cases, the remainder may be 0, it means the number is completely divisible by the divisor

Example :

```
Int a=13,b=2,c
```

```
C=a%b;
```

```
Printf("\nC=%d",c);
```

Output :

1

Assignment Work :

1. WAP to Find the Maximum/ Minimum number among the Three number Using Turnery Operator
2. WAP to check Odd / Even num of given Number

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE
DESIGN & MULTIMEDIA INSTITUTE

Ch. 8

Control Statement (Switch case)

Switch Case :

The switch statement in C is an alternate to if-else-if ladder statement which allows us to execute multiple operations for the different possible values of a single variable called switch variable. Here, We can define various statements in the multiple cases for the different values of a single variable.

Syntax :

```
switch(expression)
{
    case value1:
        statement block1;
        break;
    case value2:
        statement block2;
        break;
    case valueN:
        statement blockN;
        break;
    default:
        default statement block;
}
```

Purpose:

In switch statement, the expression gives one value which is compared with the values written with case statements. If one case is evaluated true then the statement block related to that case will be executed. Following portion of the program give the example of switch statement.

Break Keyword used for break the current case statement.

Example :

```
Void main() {
    Int n;
    Printf("Enter Day Value=");
    Scanf("%d",&n);
    Switch(n){
        Case 1: printf("Sunday");
        Case2: printf("Monday");
        Case2: printf("Tuesday");
        Case2: printf("Wednesday");
        Case2: printf("Thursday");
        Case2: printf("Friday");
        Case2: printf("Saturday");
        Default: printf("plz input valid num");
    }
}
```

Assignment work :

1. WAP to input week number and print week day
2. WAP to input month number and print number of days in that month

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

Ch. 9

Bitwise Operator

7. Bitwise Operators in C :

Bitwise operators are used to perform bit manipulations.

Generally these operators used to perform hardware related task by c programming.

Operators	Meaning
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
<<	Shift Left
>>	Shift Right

Example:

int a=7, b=5;

a & b : 111 & 101 = 101 = 5

a | b: 111 | 101 = 111 = 7

a ^ b: 111 | 101 = 010 = 2

Example :

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a=23,d,b=10,c;    // here a= 10111 B=1010
    clrscr();
    c=a&b;                // 10111&1010 = 00010
    printf("\nC=%d",c);    // 00010 = 2
    d=a|b;                // 10111 | 1010 = 11111
    printf("\nD=%d",d);    // 11111=31
    getch();
}
```

8 C – Special Operators :

Operators	Description
&	This is used to get the address of the variable. Example : &a will give address of a.
*	This is used as pointer to a variable. Example : * a where, * is pointer to the variable a.
Sizeof ()	This gives the size of the variable. Example : size of (char) will give us 1.

Size of operator :

Example :

```
Int a=20;
Printf("\nsize of A=%d",sizeof(a));
```

Output:

Size of A=2

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

CH – 10

Introduction Of Loop

While Loop

Loop Statements :

The looping can be defined as repeating the same process multiple times until a specific condition satisfies. There are three types of loops used in the C language.

Why use loops in C language?

- ✓ The looping simplifies the complex problems into the easy ones.
- ✓ It enables us to alter the flow of the program so that instead of writing the same code again and again, we can repeat the same code for a finite number of times.
- ✓ For example, if we need to print the first 10 natural numbers then, instead of using the printf statement 10 times, we can print inside a loop which runs up to 10 iterations.

Advantage of loops in C :

- 1) It provides code reusability.
- 2) Using loops, we do not need to write the same code again and again.

Types of C Loops :

There are three types of loops in C language that is given below:

while

Do while

For

While Loop :

While loop is also known as a pre-tested loop. In general, a while loop allows a part of the code to be executed multiple times depending upon a given boolean condition. It can be viewed as a repeating if statement. The while loop is mostly used in the case where the number of iterations is not known in advance

In condition you have to write a condition from relational operator then while loop execute till that condition will not false.

Syntax :

```
While(condition)
{
    statement 1;
    statement 2;
    increment/ decrement;
}
```

Assignment Work :

1. Write a program in C to display the first 10 natural numbers
2. Write a C program to print all natural numbers in reverse
3. WAP to find sum of first **N** natural numbers
4. WAP to print Range between two number
5. WAP to print Range of ODD num if first num is smallest and Print range of Even num if First num is largest
6. WAP to find factorial of a given number
7. WAP to print reverse given num
8. WAP program to count number of digits in a number

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

CH – 11

DO While Loop

Do while loop :

The do while loop is a post tested loop. Using the do-while loop, we can repeat the execution of more parts of the statements. The do-while loop is mainly used in the case where we need to execute the loop at least once. The do-while loop is mostly used in menu-driven programs where the termination condition depends upon the end user.

Syntax :

```
do{  
  //code to be executed  
}while(condition);
```

Assignment Work :

1. WAP to Display The Multiplication Table of a Given Number
2. WAP to display the cube of the number upto given an number
3. WAP program to sum of digits in a number
4. WAP to find the number and sum of all integer between two user input num which are divisible by given num
5. WAP to enter password and check password with entered password, if password is match then print "WELCOME" otherwise ask to give correct password.

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

CH – 10

Introduction Of Loop For Loop

For loop:

The for loop in C language is used to the statements or a part of the program more times. It is frequently used to traverse the data structures like the array and linked list.

Syntax:

```
for(Expression 1; Expression 2; Expression 3){  
    //code to be executed  
}
```

Assignment Work :

1. WAP to print Fibonacci series in a given range
2. WAP to check given num is Armstrong or not
3. WAP to check given num is Palindrome or not
4. WAP to check given num is Prime or not
5. WAP to print Pattern given by your Faculty (15 pattern)

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE
DESIGN & MULTIMEDIA INSTITUTE

CH – 10

Introduction Of Loop Continue | Break | Goto

BREAK STATEMENT:

- The break statement is used to terminate loops or to exit a switch.
- The break statement will break or terminate the inner-most loop.
- It can be used within a while, a do-while, a for or a switch statement.

Syntax : break;

CONTINUE STATEMENT :

- The continue statement is used to skip or to bypass some step or iteration of looping structure.
- It does not terminate the loop but just skip or bypass the particular sequence of the loop structure.

Syntax : continue ;

THE GOTO STATEMENT :

- The goto statement is used to alter the normal sequence of program execution by transferring control to some other part of the program.
- In its general form the goto statement is written as goto label; Where label is an identifier used to label the target statement to which control will be transferred.

Example :

```
#include <stdio.h>
int main()
{
    int num,i=1;
    printf("Enter the number whose table you want to print?");
    scanf("%d",&num);
    table:
    printf("%d x %d = %d\n",num,i,num*i);
    i++;
    if(i<=10)
    goto table;
}
```

Assignment Work :

1. Program to calculate the sum and average of positive numbers. If the user enters a negative number, the sum and average are displayed.
2. WAP to calculate sum of number if user enter negative num loop terminates (10 num maximum)
3. WAP to calculate sum of number if user enter negative it is not added to result (user enter maximum 10 num)

Extra Practical:

1. WAP to input basic salary of an employee and calculate its Incentive and Gross salary and net salary of given your faculty
2. WAP to create BANK Management System.

CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

CH 14

ARRAY

ARRAY :

- An array is a collection of **similar elements** of the **same data type**.
- These similar elements could be all integers, or all floats, or all characters, etc.
- Individual values of array are called as elements.
- Array can be initialized at a place where it is declared.
- Arrays are helpful to store and access a list of values under a single variable name.

Benefits of Array:

- If you want to specify more than one variable then you have to specify as follow.

```
int sub1, sub2, sub3, sub4, sub5;
```

- In place of normal variable declaration we can declare an array variable as given below.
- It means that array will be helpful to make your work easy to process.

```
int sub[5];
```

Use of Array :

- Using a single identifier name, it is possible to access many values of same type.
- Sorting & arranging multiple data related to numeric & character.
- To manage multiple values using less number of program statements.
- To store data in sequential order in memory.
- To access its element using index which always starts from 0
- It is easy to use with other programming concepts like looping structure.

Types of Array :

- (1) One-Dimensional Array or Single Dimensional Array
- (2) Two-Dimensional Array
- (3) Multi-Dimensional Array

One-Dimensional Array :

- ☐ A list of items can be given a variable name using only one subscript and such a variable is called a ***single dimensional array***.

Declaration of Array:

- ☐ Like any other variables, arrays must be declared before they are used.
- ☐ Array declaration specifies the data type of array elements like, int, float etc. and the size of the array.

■ Syntax :

DataType VariableName[size];

■ Purpose :

- ☐ To declare single dimensional array we can use above syntax.

■ Example :

int sub[6];

sub[0]	
sub[1]	
sub[2]	
sub[3]	
sub[4]	
sub[5]	

Assignment Work :

1. WAP to get the array and display it
2. WAP to display the array with the position.
3. WAP to get the array and display it in the reverse order
4. WAP to get the array and copy it into the another array
5. WAP to get the array and copy the reverse of the array into the another array
6. WAP to get two array and merge it into the third array
7. WAP Sum of the elements of the array
8. WAP Find the max value from the array
9. WAP Find the min value from the array
10. WAP to get the array and search the element from it
11. WAP to replace the particular element from the array
12. WAP to insert the element in array
13. WAP to delete the element from the array

CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

CH 15

2D ARRAY

Two-Dimensional Array

- ❑ A list of items can be store in table format by **row** and **column** is known as Two-Dimensional Array.

■ **Syntax :**

DataType VariableName[rows][columns];

■ **Purpose :**

- ❑ To declare tow dimensional array we can use above syntax.

■ **Example :**

int std[3][4]; or

int std[3][4]={10,15,20,25,30,35,40,45,50,55,60,65};

[0][0][0] 10	[0][0][1] 15
[0][1][0] 20	[0][1][1] 25
[1][0][0] 30	[1][0][1] 35
[1][1][0] 40	[1][1][1] 45

Now [0][1][0]=20
Now [1][1][0]=?

[0][0] 10	[0][1] 15	[0][2] 20	[0][3] 25
[1][0] 30	[1][1] 35	[1][2] 40	[1][3] 45
[2][0] 50	[2][1] 55	[2][2] 60	[2][3] 65

Now [0][1]=15

Now [2][3]=?

Multi-Dimensional Array :

- ❑ A list of items can be store in table format by **row** and **column** is with more then two dimension is known as **Multi-Dimensional Array**.

■ **Syntax :**

DataType VariableName[rows][col1][Col2]...;

■ **Purpose :**

- ❑ To declare multi dimensional array we can use above syntax.

■ **Example :**

```
int std[2][2][2]; or
```

```
int std[2][2][2]={10,15,20,25,30,35,40,45};
```

Assignment work :

1. WAP to get the 2-d array [matrix] and display it
2. WAP to Addition of two matrices
3. WAP to Sum of elements of the matrix
4. WAP to Find the max from the matrix
5. WAP to Find the min from the matrix
6. WAP to Row-wise sum
7. WAP to Column-wise sum

CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

CH 16 STRING

C Strings :

The string can be defined as the one-dimensional array of characters terminated by a null ('\0'). The character array or the string is used to manipulate text such as word or sentences. Each character in the array occupies one byte of memory, and the last character must always be 0. The termination character ('\0') is important in a string since it is the only way to identify where the string ends. When we define a string as char s[10], the character s[10] is implicitly initialized with the null in the memory.

There are two ways to declare a string in c language.

1. By char array
2. By string literal

Let's see the example of declaring string by char array in C language.

```
char ch[10]={ 'C', 'R', 'E', 'A', 'T', 'I', 'V', 'E', '\0'};
```

We can also define the string by the string literal in C language. For example:

```
char ch[]="CREATIVE";
```

C gets() and puts() functions :

The gets() and puts() are declared in the header file stdio.h. Both the functions are involved in the input/output operations of the strings.

Gets() -> get the string same as a scanf();

Puts() -> print the string same as a printf();

Assignment Work :

1. WAP to Enter char and display it.
2. WAP to Enter Char and display ASCII code
3. WAP to Enter decimal and display char.
4. WAP to enter name and display it.
5. WAP to enter name and count following format.
 - (i) Length of given string.
 - (ii) Total Num of alphabet.
 - (iii) Total Num of digit.
 - (iv) Total Num of Upper character.
 - (v) Total Num of Lower character.
 - (vi) Total Num of space.
 - (vii) Total Num of other character.
- 6 WAP to given string and display following format.
 - (i) Display Sentence case.
 - (ii) Display lowercase.
 - (iii) Display upper case.
 - (iv) Display title case.
 - (v) Display toggle case.
- 7 WAP to given string and find out the num of times 'a' is there in the string.
- 8 WAP to given string and Replace character in the string.
- 9 WAP to given string and Delete character in the string.
- 10 WAP to given string and display reverse order.

CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

CH 17

C In Built Library Function

C String Functions :

There are many important string functions defined in "string.h" library.

No.	Function	Description
1)	strlen(string_name)	returns the length of string name.
2)	strcpy(destination, source)	copies the contents of source string to destination string.
3)	strcat(first_string,Second_string)	concatenates or joins first string with second string. The result of the string is stored in first string.
4)	strcmp(first_string,second_string)	compares the first string with second string. If both strings are same, it returns 0.
5)	strrev(string)	returns reverse string.
6)	strlwr(string)	returns string characters in lowercase.
7)	strupr(string)	returns string characters in uppercase.

Maths Library Function :

The **math.h** header defines various mathematical functions and one macro. All the functions available in this library take **double** as an argument and return **double** as the result. Let us discuss some important functions one by one.

#include<math.h>

1) ceil()

Syntax : ceil(double value);

Purpose:

To round a value with higher integer value. We can use **ceil** function.

Example :

```
double a=2.5;
printf("Ceil value of %lf is %0.0lf", a, ceil(a));
```

Output :

Ceil value of **2.5** is **3**

2) floor()

Syntax : floor(double value);

Purpose:

To round a value with lower integer value. We can use **floor** function.

Example :

```
double a=2.5;
printf("Floor value of %lf is %0.0lf", a, floor(a));
```

Output :

Ceil value of **2.5** is **2**

3) pow()

Syntax : double pow(double, double);

Purpose:

To find power of value we can use **pow** function.

Example :

```
double x = 2.0, y = 3.0;  
printf("Ans =2lf",pow(x, y));
```

Output :

Ans =8.00

4) sqrt()

Syntax : double sqrt(double);

Purpose:

To find square root value. We can use **sqrt** function.

Example :

```
double x = 4.0;  
printf("Square root of %lf is %lf", x, sqrt(x));
```

Output :

Square root of 4.00 is 2.00

Extra Functions : <dos.h>

1) delay()

Syntax : delay(milliseconds);

Purpose:

To hold the process in milliseconds. We can use **delay** function.

Example :

```
printf("Hi! Friends");
```

```
delay(5000);  
printf("\nWelCome");
```

2) isalpha()

Syntax : isalpha(char);

Purpose:

To check that given characters are **Alphabet or Not**. We can use **isalpha** function.

Example :

```
char val[]="CREATIVE";  
if(isalpha(val))  
    printf("Given String is ALPHA");  
else  
    printf("Given String is NOT ALPHA");
```

3) isalnum()

Syntax : isalnum(char);

Purpose:

To check that given characters are **Alpha numeric or Not**. We can use **isalnum** function.

Example :

```
char val[]="123ABC";  
if(isalnum(val))  
    printf("Given String is ALNUM");  
else  
    printf("Given String is NOT ALNUM");
```

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

CH - 18

USER DEFINED FUNCTION

C Functions

- In c, we can divide a large program into the basic building blocks known as function.
- The function contains the set of programming statements enclosed by {}.
- A function can be called multiple times to provide reusability and modularity to the C program.
- In other words, we can say that the collection of functions creates a program.

Advantage of functions in C

There are the following advantages of C functions.

- By using functions, we can avoid rewriting same logic/code again and again in a program.
- We can call C functions any number of times in a program and from any place in a program.
- We can track a large C program easily when it is divided into multiple functions.
- Reusability is the main achievement of C functions.

Types of Functions

There are two types of functions in C programming:

1. **Library Functions:** are the functions which are declared in the C header files such as scanf(), printf(), gets(), puts(), ceil(), floor() etc.
2. **User-defined functions:** are the functions which are created by the C programmer, so that he/she can use it many times. It reduces the complexity of a big program and optimizes the code.

User Defined Functions (UDF) :

- ❑ Function which are defined by USER.
- ❑ Function which are define as per user requirement are known as **User Defined Function**.
- ❑ UDF are also used to stop repetition and duplication of codes.
- ❑ While working with user defined function, first of all we have to declare the function and its return type.
- ❑ If your function will not return any value then declare function with VOID type.
- ❑ If your data will be returned in the form of integer then we have to declare UDF with int data type.
- ❑ If your data will be returned in the form of float then we have to declare UDF float data type.

How to define UDF ?

Syntax:

ReturnType FunctionName(argument list);

Purpose:

To declare UDF with required return type we can use above syntax.

Example :

```
void starline( );  
int sum(int,int);  
float interest(float,float,float);
```

Types of User Defined Functions :

The function can categories in four types. As per its argument and return type.

- [1] No Argument and No Return Value.
- [2] With Argument and No Return Value.
- [3] With argument and With Return Value.
- [4] No Argument and With Return Value.

1) No Argument and No Return Value :

- ✓ When function doesn't have any argument and doesn't have any return value at that time it is called as *Function with no Argument and no Return Value*.
- ✓ Means the called function will not receive any argument from the calling function and it will not return any value to calling function.
- ✓ We can use any UDF at any number of times.

2) With Argument and No Return Value :

- ✓ When function have some argument to send called function and have not any return value at that time it is called as *Function with Argument and No Return Value*.
- ✓ Means the called function will receive some argument from the calling function but it will no return value to calling function.

3) With argument and With Return Value :

- ✓ When function have some argument to send called function and have return value at that time it is called as *Function with Argument and Return Value*.
- ✓ Means the called function will receive some argument from the calling function and it will return value to calling function.

4) No Argument and With Return Value :

- ✓ When function doesn't have any argument and have any return value at that time it is called as *Function with no Argument and Return Value*.
- ✓ Means the called function will not receive any argument from the calling function but it will return value to calling function.

Assignment:

1. WAP to enter two value and that values of sum,sub,mul and div using UDF.
2. WAP to area of triangle, circle, rectangle and square with switch case.
3. WAP to find out factorial of a given num using function.
4. WAP to interchange values of two variable using function and global Variable
5. WAP to find out maximum out of three num using function.
6. WAP to reverse a string using function.
7. WAP to sort a list of names in alphabetical order using function.

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

CH - 19

Recursion

Recursion:

- ✓ Any Function can also call to itself.
- ✓ When any function call it self it is called as recursive function (recursion).
- ✓ It will work till the specified condition will satisfied.
- ✓ **Some points to remember about Recursive Function are as follow.**
 1. Recursive function can be used as a loop.
 2. The function is terminated when any particular condition is not satisfied.
 3. If a recursive function returns some value, it starts returning a value at last when the condition is not satisfied.

Example :

In the following example, recursion is used to calculate the factorial of a number.

```
#include <stdio.h>
int fact(int);
int main()
{
    int n,f;
    printf("Enter the number whose factorial you want to calculate?");
    scanf("%d",&n);
    f = fact(n);
    printf("factorial = %d",f);
}
int fact(int n) {
    if (n==0) {
        return 0;
    } else if ( n == 1) {
        return 1;
    } else {
        return n*fact(n-1);
    }
}
```

Output

Enter the number whose factorial you want to calculate?5
factorial = 120

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

CH - 20

Structure

Union

Structure in C :

- ✓ **Structure in c language** is a *user defined datatype* that allows you to hold different type of elements.
- ✓ Each element of a structure is called a member.
- ✓ It works like a template in C++ and class in Java. You can have different type of elements in it.
- ✓ It is widely used to store student information, employee information, product information, book information etc.

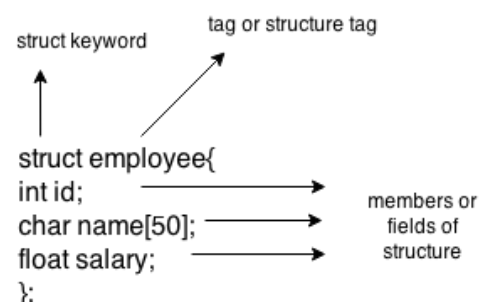
Defining structure :

The **struct** keyword is used to define structure. Let's see the syntax to define structure in c.

```
struct structure_name
{
    data_type member1;
    data_type member2;
    .
    .
    data_type memberN;
};
```

Let's see the example to define structure for employee in c.

```
struct employee
{ int id;
  char name[50];
  float salary;
};
```



Here, **struct** is the keyword, **employee** is the tag name of structure; **id**, **name** and **salary** are the members or fields of the structure. Let's understand it by the diagram.

Example : example of structure in C language.

```
#include <stdio.h>
#include <string.h>
struct employee
{
    int id;
    char name[50];
}e1; //declaring e1 variable for structure
int main( )
{
    //store first employee information
    e1.id=101;
    strcpy(e1.name, "keyur patel "); //copying string into char array
    //printing first employee information
    printf( "employee 1 id : %d\n", e1.id);
    printf( "employee 1 name : %s\n", e1.name);
    return 0;
}
```

Output:

```
employee 1 id : 101
employee 1 name : keyur patel
```

C Union :

Like structure, **Union in c language** is a *user defined datatype* that is used to hold different type of elements.

But it doesn't occupy sum of all members size. It occupies the memory of largest member only. It shares memory of largest member.

<u>Structure</u>	<u>Union</u>
<pre>struct Employee{ char x; // size 1 byte int y; //size 2 byte float z; //size 4 byte }e1; //size of e1 = 7 byte</pre>	<pre>union Employee{ char x; // size 1 byte int y; //size 2 byte float z; //size 4 byte }e1; //size of e1 = 4 byte</pre>
size of e1= 1 + 2 + 4 = 7	size of e1= 4 (maximum size of 1 element)

Advantage of union over structure :

It **occupies less memory** because it occupies the memory of largest member only.

Defining union :

The **union** keyword is used to define union. Let's see the syntax to define union in c.

```
union union_name
{
    data_type member1;
    data_type member2;
    .
    .
    data_type memberN;
};
```

Example :

```
#include <stdio.h>
#include <string.h>
union employee
{
    int id;
    char name[50];
}e1; //declaring e1 variable for union
int main( )
{
    //store first employee information
    e1.id=101;
    strcpy(e1.name, " keyur patel "); //copying string into char array
    //printing first employee information
    printf( "employee 1 id : %d\n", e1.id);
    printf( "employee 1 name : %s\n", e1.name);
    return 0;
}
```

Output:

```
employee 1 id : 1869508435
employee 1 name : keyur patel
```

As you can see, id gets garbage value because name has large memory size. So only name will have actual value.

AN ISO 9001:2015 CERTIFIED INSTITUTE



CREATIVE

DESIGN & MULTIMEDIA INSTITUTE

CH - 21

Type OF Variable

Pre-Processor

Types of Variables :

1) Local Variable :

A variable that is declared inside the function or block is called a local variable. It must be declared at the start of the block.

```
void function1(){  
    int x=10;//local variable  
}
```

You must have to initialize the local variable before it is used.

2) Global Variable :

A variable that is declared outside the function or block is called a global variable. Any function can change the value of the global variable. It is available to all the functions.

It must be declared at the start of the block.

```
int value=20;//global variable  
void function1(){  
    int x=10;//local variable  
}
```

3) Automatic Variable :

All variables in C that are declared inside the block, are automatic variables by default. We can explicitly declare an automatic variable using auto keyword.

```
void main(){  
    int x=10;//local variable (also automatic)  
    auto int y=20;//automatic variable  
}
```

4) Static Variable :

A variable that is declared with the static keyword is called static variable.

It retains its value between multiple function calls.

```

void function1(){
int x=10;//local variable
static int y=10;//static variable
x=x+1;
y=y+1;
printf("%d,%d",x,y);
}

```

If you call this function many times, the local variable will print the same value for each function call, e.g. 11,11,11 and so on. But the static variable will print the incremented value in each function call, e.g. 11, 12, 13 and so on.

5 External Variable :

We can share a variable in multiple C source files by using an external variable. To declare an external variable, you need to use extern keyword.

myfile.h

```
extern int x=10;//external variable (also global)
```

program1.c

```
#include "myfile.h"
```

```
#include <stdio.h>
```

```
void printValue(){
```

```
    printf("Global variable: %d", global_variable);
```

```
}
```

C Pre-Processor :

- What is C preprocessor?
 - The C preprocessor is the first part of the compilation process.
- C pre-processor are predefined directives.
- C preprocessor lines beginning with '#'.

No	Directive	Use
1	#include	Specifies the files to be included
2	#define	Defines a macro substitution
3	#undef	Undefines a macro
4	#ifdef	Test for a macro definition
5	#endif	Specifies the end of #if
6	#if	Test a compile-time condition
7	#else	Specifies alternatives when #if test fails

C #include :

The #include pre-processor directive is used to paste code of given file into current file. It is used include system-defined and user-defined header files. If included file is not found, compiler renders error.

By the use of #include directive, we provide information to the pre-processor where to look for the header files. There are two variants to use #include directive.

1. #include <filename>
2. #include "filename"

The **#include <filename>** tells the compiler to look for the directory where system header files are held..

The **#include "filename"** tells the compiler to look in the current directory from where program is running.

#define :

The #define pre-processor directive is used to define constant or micro substitution. It can use any basic data type.

Syntax : **#define token value**

Let's see an example of #define to define a constant.

```
#include <stdio.h>
#define PI 3.14
main() {
    printf("%f",PI);
}
Output: 3.140000
```

C #undef :

The #undef preprocessor directive is used to undefine the constant or macro defined by #define.

Syntax: **#undef token :**

Let's see a simple example to define and undefine a constant.

```
#include <stdio.h>
#define PI 3.14
main() {
    printf("%f",PI);
    #undef PI
    printf("%f",PI);
}
Output: Compile Time Error: 'PI' undeclared
```

The #undef directive is used to define the preprocessor constant to a limited scope so that you can declare constant again.

Let's see an example where we are defining and undefining number variable. But before being undefined, it was used by square variable.

```
#include <stdio.h>
#define number 15
int square=number*number;
#undef number
main() {
    printf("%d",square);
}
```

Output: 225

C #ifdef :

The `#ifdef` preprocessor directive checks if macro is defined by `#define`. If yes, it executes the code otherwise `#else` code is executed, if present.

Syntax:

```
#ifdef MACRO
    //code
#endif
```

simple example to use `#ifdef` preprocessor directive.

```
#include <stdio.h>
#include <conio.h>
#define NOINPUT
void main() {
    int a=0;
    #ifdef NOINPUT
        a=2;
    #else
        printf("Enter a:");
        scanf("%d", &a);
    #endif
    printf("Value of a: %d\n", a);
}
```

Output: Value of a: 2

But, if you don't define `NOINPUT`, it will ask user to enter a number.

Example 2:

```
void main() {
    int a=0;
    #ifdef NOINPUT
        a=2;
    #else
        printf("Enter a:");
        scanf("%d", &a);
    #endif
    printf("Value of a: %d\n", a);
}
```

Output: Enter a:5
 Value of a: 5

C #if

The #if preprocessor directive evaluates the expression or condition. If condition is true, it executes the code otherwise #elseif or #else or #endif code is executed.

Syntax:

```
#if expression
    //code
#endif
```

Let's see a simple example to use #if #else preprocessor directive.

```
#include <stdio.h>
#include <conio.h>
#define NUMBER 1
void main()
{
    #if NUMBER==0
        printf("Value of Number is: %d",NUMBER);
    #else
        print("Value of Number is non-zero");
    #endif
    getch();
}
```

CH – 22

POINTER

POINTER :

- The pointer in C language is a variable, it is also known as locator or indicator that points to an address of a value.
- A Pointer in C language is a variable which holds the address of another variable of same data type.

Address in C

- Whenever a variable is defined in C language, a memory location is assigned for it, in which it's value will be stored.
- We can easily check this memory address, using the & symbol.
- If var is the name of the variable, then &var will give it's address.

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
    int var = 7;
```

```
    printf("Value of the variable var is: %d\n", var);
```

```
    printf("Memory address of the variable var is: %x\n", &var);
```

```
}
```

Output:

Value of the variable var is: 7

Memory address of the variable var is: bcc7a00

Advantage of pointer :

- Pointers are more efficient in handling Arrays and Structures.
- Pointers allow references to function and thereby helps in passing of function as arguments to other functions.
- It reduces length of the program and its execution time as well.
- It allows C language to support Dynamic Memory management.

Usage of pointer :

There are many usage of pointers in c language.

1) Dynamic memory allocation

In c language, we can dynamically allocate memory using malloc() and calloc() functions where pointer is used.

2) Arrays, Functions and Structures

Pointers in c language are widely used in arrays, functions and structures. It reduces the code and improves the performance.

Declaration of C Pointer variable:

The general syntax of pointer declaration is,

datatype *pointer_name;

The data type of the pointer and the variable to which the pointer variable is pointing must be the same.

Initialization of C Pointer variable:

- Pointer Initialization is the process of assigning address of a variable to a pointer variable.
- address operator & is used to determine the address of a variable.

```
int a = 10;
```

```
int *ptr;    //pointer declaration
```

```
ptr = &a; //pointer initialization
```

Pointer variable always points to variables of the same datatype. For example:

```
float a;
```

```
int *ptr = &a;    // ERROR, type mismatch
```

A pointer that is assigned a NULL value is called a NULL pointer in C.

```
int *ptr = NULL;
```

Example:

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int a;
```

```
    a = 10;
```

```
    int *p = &a;    // declaring and initializing the pointer
```

```
    //prints the value of 'a'
```

```
    printf("%d\n", *p);
```

```
    printf("%d\n", *&a);
```

```
    //prints the address of 'a'
```

```
    printf("%u\n", &a);
```

```
    printf("%u\n", &p);    //prints address of 'p'
```

```
}
```

Pointer to Array

- When an array in C language is declared, compiler allocates sufficient memory to contain all its elements. Its base address is also allocated by the compiler.
- Use a pointer to an array, and then use that pointer to access the array elements.

```
int arr[5] = { 1, 2, 3, 4, 5 };
```

arr has two purpose -

- It is the name of the array
- It acts as a pointer pointing towards the first element in the array

arr is equal to &arr[0] by default

Example:

```
#include<stdio.h>

void main()
{
    int a[3] = {1, 2, 3};
    int *p = a;
    for (int i = 0; i < 3; i++)
    {
        printf("%d", *(p+i));
    }
}
```

Output: 1 2 3

Syntax:

`*(a+i) //pointer with an array`

is same as:

`a[i]`

Pointer to Multidimensional Array

- In `a[i][j]`, `a` will give the base address of this array, even `a + 0 + 0` will also give the base address, that is the address of `a[0][0]` element.

Syntax:

`*(*(a + i) + j)`

Pointer and Character strings

- Pointer is used to create strings. Pointer variables of `char` type are treated as string.

`char *str = "Hello";`

The content of the string can be printed using `printf()` and `puts()`.

`printf("%s", str);`

`puts(str);`

Array of Pointers—Pointer within Array

- Pointers are very helpful in handling character arrays with rows of varying lengths.
- “Array of pointers” is an array of the pointer variables. It is also known as pointer arrays.

Syntax:

`int *var_name[array_size];`

Example:

```
char *name[3] = { "Adam","chris","Deniel"};
```

```
int *ptr[3];
```

// example of array of pointers.

```
#include <stdio.h>
```

```
int SIZE = 3;
```

```
void main()
```

```
{
```

```
    int arr[] = { 1, 2, 3 };
```

```
    int i, *ptr[SIZE];
```

```
    for (i = 0; i < SIZE; i++) {
```

```
        ptr[i] = &arr[i]; // assigning the address of integer.
```

```
    }
```

```
    // printing values using pointer
```

```
    for (i = 0; i < SIZE; i++) {
```

```
        printf("Value of arr[%d] = %d\n", i, *ptr[i]);
```

```
    }
```

```
}
```

Output:

Value of arr[0] = 1

Value of arr[1] = 2

Value of arr[2] = 3

Example:

an array of pointers to the character to store a list of strings.

```
#include <stdio.h>
```

```
int size = 4;
```

```
void main()
```

```
{
```

```
    // array of pointers to a character to store a list of strings
```

```
    char* names[] = {"amit","amar","ankit","akhil"};
```

```
    int i = 0;
```

```
    for (i = 0; i < size; i++) {
```

```
        printf("%s\n", names[i]);
```

```
    }
```

```
}
```

Output:

amit

amar

ankit

akhil

What is a Pointer to an Array?

The array pointer is an alternative name to a pointer to an array. We generally make use of this pointer for accessing the various components of any given array. The pointer ptr basically focuses on the 0th component of any given array. Likewise, one can easily declare a point that is capable of pointing to a whole array- rather than only a single array component.

What is an Array of Pointers?

It refers to an array of various pointer variables, also called the pointer arrays. One can easily create separate pointer variables pointing towards different ranges of values. And we can also create a single integer array of pointers pointing towards all the available values.

Difference between a Pointer to an Array and Array of Pointers

Here is a list of the differences present between Pointer to an Array and Array of Pointers.

Parameters	Pointer to an Array	Array of Pointers
Uses and Purposes	A user creates a pointer for storing the address of any given array.	A user creates an array of pointers that basically acts as an array of multiple pointer variables.
Alternative Names	It is alternatively known as an array pointer.	These are alternatively known as pointer arrays.
Allocation	One can allocate these during the run time.	One can allocate these during the compile time.
Initialization at Definition	You cannot initialize a pointer to the definition.	You can easily initialize an array at the definition level.
Nature	It is dynamic in nature.	It is static in nature.
Resizing	One can easily resize the allocated memory of a pointer later at any given time.	Once we declare the size of an array, we cannot resize it any time we want according to our requirements.
Type of Storage	A typical pointer variable is capable of storing only a single variable within.	The size of any given array decides the total number of variables that it can store within.

Pointer to Structures

We can also have pointer to a single structure variable, but it is mostly used when we are dealing with array of structure variables.

```
#include <stdio.h>
```

```
struct Book
```

```
{
```

```
    char name[10];
```

```
    int price;
```

```
}
```

```
void main()
```

```
{
```

```
    struct Book a;    //Single structure variable
```

```
    struct Book* ptr; //Pointer of Structure type
```

```
    ptr = &a;
```

```
    struct Book b[10]; //Array of structure variables
```

```
    struct Book* p;    //Pointer of Structure type
```

```
    p = &b;
```

```
}
```

Accessing Structure Members with Pointer

To access members of structure using the structure variable, we used the dot . operator. But when we have a pointer of structure type, we use arrow -> to access structure members.

Example:

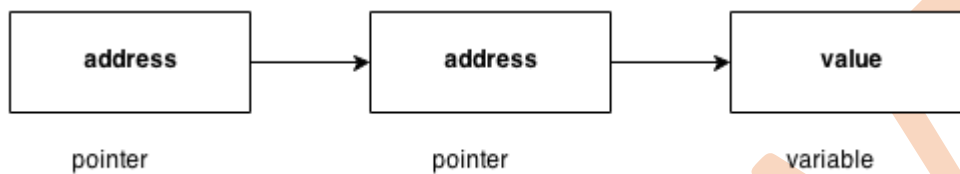
```
#include <stdio.h>
```

```
struct my_structure {  
    char name[20];  
    int number;  
    int rank;  
};
```

```
void main()  
{  
    struct my_structure var = {"Creative", 35, 1};  
    struct my_structure *ptr;  
    ptr = & var;  
  
    printf("NAME: %s\n", ptr->name);  
    printf("NUMBER: %d\n", ptr->number);  
    printf("RANK: %d", ptr->rank);  
}
```

Pointer to a Pointer in C(Double Pointer)

- Pointers are used to store the **address of other variables** of similar datatype.
- if you want to store the **address of a pointer variable**, then you again need a pointer to store it.
- Thus, when one pointer variable stores the address of another pointer variable, it is known as **Pointer to Pointer variable or Double Pointer**.



Syntax:

```
int **p1;
```

Simple program to represent Pointer to a Pointer

```
#include <stdio.h>
```

```
void main() {
```

```
    int a = 10;
```

```
    int *p1;    //this can store the address of variable a
```

```
    int **p2;    //    this can store the address of pointer variable p1 only.
```

```
    p1 = &a;
```

```
    p2 = &p1;
```

```
    printf("Address of a = %u\n", &a);
```

```
    printf("Address of p1 = %u\n", &p1);
```

```
    printf("Address of p2 = %u\n\n", &p2);
```

```
// below print statement will give the address of 'a'
printf("Value at the address stored by p2 = %u\n", *p2);
printf("Value at the address stored by p1 = %d\n\n", *p1);

printf("Value of **p2 = %d\n", **p2); //read this *(*p2)
}
```

Output:

Address of a = 2686724

Address of p1 = 2686728

Address of p2 = 2686732

Value at the address stored by p2 = 2686724

Value at the address stored by p1 = 10

Value of **p2 = 10

Dynamic Memory Allocation

- If there is a situation where only 5 elements are needed to be entered in this array. In this case, the remaining 4 indices are just wasting memory in this array. So there is a requirement to lessen the length (size) of the array from 9 to 5.
- Take another situation. In this, there is an array of 9 elements with all 9 indices filled. But there is a need to enter 3 more elements in this array. In this case, 3 indices more are required. So the length (size) of the array needs to be changed from 9 to 12.

Dynamic Memory Allocation can be defined as a procedure in which the size of a data structure (like Array) is changed during the runtime.

An array is a collection of a fixed number of values. Once the size of an array is declared, you cannot change it.

Sometimes the size of the array you declared may be insufficient. To solve this issue, you can allocate memory manually during run-time. This is known as **dynamic memory allocation** in C programming.

1. C malloc()

- The name "malloc" stands for memory allocation.
- The malloc() function reserves a block of memory of the specified number of bytes. And, it returns a pointer of void which can be casted into pointers of any form.

Syntax of malloc()

```
ptr = (castType*) malloc(size);
```

Example

```
ptr = (float*) malloc(100 * sizeof(float));
```

The above statement allocates 400 bytes of memory. It's because the size of float is 4 bytes. And, the pointer ptr holds the address of the first byte in the allocated memory.

2. C calloc()

- The name "calloc" stands for contiguous allocation.
- The malloc() function allocates memory and leaves the memory uninitialized, whereas the calloc() function allocates memory and initializes all bits to zero.

Syntax of calloc()

```
ptr = (castType*)calloc(n, size);
```

Example:

```
ptr = (float*) calloc(25, sizeof(float));
```

The above statement allocates contiguous space in memory for 25 elements of type float.

3. C free()

- Dynamically allocated memory created with either calloc() or malloc() doesn't get freed on their own.
- You must explicitly use free() to release the space.

Syntax of free()

```
free(ptr);
```

This statement frees the space allocated in the memory pointed by ptr.

Example: malloc() and free()

// Program to calculate the sum of n numbers entered by the user

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void main() {
```

```
    int n, i, *ptr, sum = 0;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    ptr = (int*) malloc(n * sizeof(int));
```

```
    printf("Enter elements: ");
```

```
    for(i = 0; i < n; ++i) {
```

```
        scanf("%d", ptr + i);
```

```
        sum += *(ptr + i);
```

```
    }
```

```
    printf("Sum = %d", sum);
```

```
    free(ptr);           // deallocating the memory
```

```
}
```

Output

Enter number of elements: 3

Enter elements: 100

20

36

Sum = 156

Example: calloc() and free()**// Program to calculate the sum of n numbers entered by the user**

```
#include <stdio.h>
#include <stdlib.h>

void main() {
    int n, i, *ptr, sum = 0;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    ptr = (int*) calloc(n, sizeof(int));
    printf("Enter elements: ");
    for(i = 0; i < n; ++i) {
        scanf("%d", ptr + i);
        sum += *(ptr + i);
    }
    printf("Sum = %d", sum);
    free(ptr);
    return 0;
}
```

Output:

Enter number of elements: 3

Enter elements: 100

20

36

Sum = 156

4. C realloc()

- If the dynamically allocated memory is insufficient or more than required, you can change the size of previously allocated memory using the realloc() function.

Syntax of realloc()

```
ptr = realloc(ptr, x);
```

Here, ptr is reallocated with a new size x.

Example: realloc()

```
#include <stdio.h>
```

```
void main () {
```

```
    char *ptr;
```

```
    ptr = (char *) malloc(10);
```

```
    strcpy(ptr, "Programming");
```

```
    printf(" %s, Address = %u\n", ptr, ptr);
```

```
    ptr = (char *) realloc(ptr, 20); //ptr is reallocated with new size
```

```
    strcat(ptr, " In 'C'");
```

```
    printf(" %s, Address = %u\n", ptr, ptr);
```

```
    free(ptr);
```

```
}
```

CH – 23

File Handling

File Handling in C

File is a collection of bytes that is stored on secondary storage devices like disk.

BASIC FILE OPERATIONS IN C PROGRAMMING:

There are 4 basic operations that can be performed on any files in C programming language. They are,

1. Opening/Creating a file
2. Closing a file
3. Reading a file
4. Writing in a file

fopen() – To open a file

Declaration: `FILE *fopen (const char *filename, const char *mode)`

fopen() function is used to open a file to perform operations such as reading, writing etc.

`FILE *fp;`

`fp=fopen ("filename", "'mode");`

Where,

fp – file pointer to the data type "FILE".

filename – the actual file name with full path of the file.

mode – refers to the operation that will be performed on the file.

- **r** – Opens a file in read mode and sets pointer to the first character in the file. It returns null if file does not exist.
- **w** – Opens a file in write mode. It returns null if file could not be opened. If file exists, data are overwritten.
- **a** – Opens a file in append mode. It returns null if file couldn't be opened.
- **r+** – Opens a file for read and write mode and sets pointer to the first character in the file.
- **w+** – opens a file for read and write mode and sets pointer to the first character in the file.
- **a+** – Opens a file for read and write mode and sets pointer to the first character in the file. But, it can't modify existing contents.

fclose() – To close a file

Declaration: `int fclose(FILE *fp);`

fclose() function closes the file that is being pointed by file pointer fp. In a C program, we close a file as below.

```
fclose (fp);
```

fgets() – To read a file

Declaration: `char *fgets(char *string, int n, FILE *fp)`

fgets function is used to read a file line by line. In a C program, we use fgets function as below.

fgets (buffer, size, fp);

where,

buffer – buffer to put the data in.

size – size of the buffer

fp – file pointer

fprintf() – To write into a file

Declaration: `int fprintf(FILE *fp, const char *format, ...);`

fprintf() function writes string into a file pointed by fp. In a C program, we write string into a file as below.

```
fprintf (fp, "some data"); or
```

```
fprintf (fp, "text %d", variable_name);
```

fscanf() - reads formatted data from a file**Declaration:** `int fscanf(FILE *fp, const char *format, ...)`

fscanf() function is used to read formatted data from a file.

```
fscanf (fp, "%d", &age);
```

Where, fp is file pointer to the data type "FILE".

Age – Integer variable

ftell() - gives current position of file pointer**Declaration:** `long int ftell(FILE *fp)`

ftell function is used to get current position of the file pointer. In a C program, we use ftell() as below.

```
ftell(fp);
```

rewind () - moves file pointer position to the beginning of the file**Declaration:** `void rewind(FILE *fp)`

rewind function is used to move file pointer position to the beginning of the file. In a C program, we use rewind() as below.

```
rewind(fp);
```

EXAMPLE PROGRAM FOR FILE OPEN, FILE WRITE AND FILE CLOSE IN C LANGUAGE:

```
# include <stdio.h>
```

```
# include <string.h>
```

```
void main( )
```

```
{
```

```
    FILE *fp ;
```

```
    char data[50];
```

```
    printf( "Opening the file test.c in write mode" ) ;
```

```
    fp = fopen("test.c", "w") ;
```

```

if ( fp == NULL )
{
    printf( "Could not open file test.c" ) ;
}

printf( "\n Enter some text from keyboard to write in the file test.c" ) ;

while ( strlen (gets(data)) > 0 )
{
    fputs(data, fp) ;
    fputs("\n", fp) ;
}

printf("Closing the file test.c") ;
fclose(fp) ;
}

```

OUTPUT:

Opening the file test.c in write mode

Enter some text from keyboard to write in the file test.c

Hai, How are you?

Closing the file test.c

EXAMPLE PROGRAM FOR FILE OPEN, FILE READ AND FILE CLOSE IN C LANGUAGE:

```
# include <stdio.h>
```

```
void main( )
```

```
{
```

```
    FILE *fp ;
```

```
    char data[50] ;
```

```
    printf( "Opening the file test.c in read mode" ) ;
```

```

fp = fopen( "test.c", "r" );
if ( fp == NULL )
{
    printf( "Could not open file test.c" );
}
printf( "Reading the file test.c" );
while( fgets ( data, 50, fp ) != NULL )
printf( "%s" , data );
printf("Closing the file test.c");
fclose(fp);
}

```

OUTPUT:

Opening the file test.c in read mode

Reading the file test.c

Hai, How are you?

Closing the file test.c

EXAMPLE PROGRAM FOR FSCANF(), FPRINTF(), FTELL(), REWIND() FUNCTIONS

```

#include <stdio.h>
void main ()
{
    char name [20];
    int age, length;
    FILE *fp;
    fp = fopen ( "test.txt", "w");
    fprintf (fp, "%s %d", "Creative", 5);
    length = ftell(fp); // Cursor position is now at the end of file
    rewind (fp); // It will move cursor position to the beginning of the file

```

```
fscanf (fp, "%d", &age);
fscanf (fp, "%s", name);
fclose (fp);
printf ("Name: %s \n Age: %d \n",name,age);
printf ("Total number of characters in file is %d", length);
}
```

Output:

Name: Creative

Age: 5

Total number of characters in file is 15

getw() - reads an integer from file

Declaration: `int getw(FILE *fp);`

getw function reads an integer value from a file pointed by fp. In a C program, we can read integer value from a file as below.

```
getw(fp);
```

putw () - writes an integer to file

Declaration: `int putw(int number, FILE *fp);`

putw function is used to write an integer into a file. In a C program, we can write integer value in a file as below.

```
putw(i, fp);
```

where

i – integer value

fp – file pointer

EXAMPLE PROGRAM FOR PUTW(), GETW() FUNCTION

```
#include <stdio.h>
```

```
void main ()
```

```
{
```

```
    FILE *fp;
```

pg. 7

```
int i=1, j=2, k=3, num;
fp = fopen ("test.c","w");
putw(i,fp);
putw(j,fp);
putw(k,fp);
fclose(fp);
fp = fopen ("test.c","r");
while(getw(fp)!=EOF)
{
    num= getw(fp);
    printf("Data in test.c file is %d \n", num);
}
fclose(fp);
}
```

Output:

Data in test.c file is

1
2
3

fseek () - moves file pointer position to given location

Declaration: int fseek(FILE *fp, long int offset, int whence)

fseek() function is used to move file pointer position to the given location.

where,

fp – file pointer

offset – Number of bytes/characters to be offset/moved from whence/the current file pointer position

whence – This is the current file pointer position from where offset is added. Below 3 constants are used to specify this.

SEEK_SET – It moves file pointer position to the beginning of the file.

SEEK_CUR – It moves file pointer position to given location.

SEEK_END – It moves file pointer position to the end of file.

CREATIVE